

Lab7

ส่วนที่ 1: Software Design & Principles Explanation

อธิบายสถาปัตยกรรมและ GRASP Patterns:

สถาปัตยกรรม:

- **Entity:** เก็บโครงสร้างข้อมูลสำหรับ Game โดยจะ mapping ไปยัง database และ ยังเพิ่ม logic เพิ่มเติมได้ด้วย
- **Repository:** เข้าถึงข้อมูลใน database โดย เป็นตัวกลาง database(model) และ service
- **Service:** สำหรับทำ business logic ต่างๆ
- **Controller:** สำหรับรับ Request และส่ง Response/View

GRASP Patterns:

- **Controller Pattern:** คือการกำหนด Object ที่ทำหน้าที่ รับ Request จากภายนอกระบบ แล้วส่งต่อให้ Object ที่เหมาะสมทำงาน และ เรียกใช้ service
เช่น @GetMapping("/"), @PostMapping("/")
- **High Cohesion:** 1 class ทำได้เพียง 1 หน้าที่ เช่น gameService จะจัดการแค่ business logic ของ game เท่านั้น
- **Low Coupling:** แต่ละ class ไม่ควรรู้รายละเอียด class อื่นมากเกินไป เช่น gameService รู้แค่การจัดการ business logic แต่ไม่ต้องรู้ว่า controller จะดึงไปใช้อย่างไร
- **Information Expert:** ให้ความรับผิดชอบกับ Class ที่มีข้อมูลที่จำเป็นที่สุดในการทำงานนั้น
- **Indirection:** คือการใช้ Object ตัวกลางเพื่อ ลดการพึ่งพากันโดยตรง เช่น service เป็นตัวกลางระหว่าง repository กับ controller

อธิบาย High-Level SOLID Principles:

- **SRP:** หนึ่ง Class ควรมีหน้าที่หลักเพียงอย่างเดียว เช่น
 - Entity: จัดการโครงสร้างข้อมูล
 - Repository: จัดการหน้าที่สำหรับ database
 - Service: จัดการ business logic
 - Controller: จัดการ HTTP req, res
- **OCP:** เป็นการเพิ่มโค้ดหรือ function ใหม่โดยไม่ต้องแก้ไขส่วนเดิมมาก เช่น
 - DiscountStrategy เป็น interface เพียงแค่ เพิ่ม new strategy และ implement ใหม่
- **LSP:** Class ลูกควรสามารถนำมาแทน Class แม่ หรือ Interface ที่ Implement ได้ โดยระบบยังทำงานถูกต้อง เช่น
 - DiscountStrategy สามารถใช้ = new NoDiscountStrategy, SeasonalSaleStrategy หรือ StudentDiscountStrategy ได้เลย
- **ISP:** ไม่ควรบังคับให้ Class ต้อง Implement Method ที่ตัวเองไม่ได้ใช้งาน เช่น
 - interface DiscountStrategy มีแค่ method เดียว class อื่นเลยนำไปใช้ได้แค่ที่จำเป็น
- **DIP:** Class ระดับสูงไม่ควรขึ้นอยู่กับ Class ระดับต่ำโดยตรง แต่ควรขึ้นอยู่กับ Abstraction
 - ไม่สร้าง = new GameRepository() แต่จะทำการผ่าน constructor หรือใช้ @Autowired

อธิบาย Strategy Pattern:

- DiscountStrategy เป็น interface และมี method ได้แก่
 - calculate()
- Implements ให้แก่ NoDiscountStrategy, SeasonalSaleStrategy, StudentDiscountStrategy
- DiscountContext ทำหน้าที่สำหรับเช็ค discountType แล้วจะเลือกใช้ตาม type ที่เลือก

ประโยชน์:

- ลดการเขียนโค้ดเพิ่ม
- เพิ่ม type อื่นได้ง่าย
- แยก logic ของแต่ละ Strategy

อธิบาย Layered Architecture:

- ทำไมต้องแยก Service Layer ออกจาก Controller และ Repository
 - เพราะว่า service เป็นตัวกลาง ระหว่าง controller & repository, service ใช้ทำ logic ต่างๆ, repository ใช้ทำการคุยหรือเข้าถึง database เพียงเท่านั้น, controller ใช้ทำ http req,res
- ประโยชน์ด้าน Low Coupling และ High Cohesion
 - Low Coupling: service ไม่ต้องคุยกับ database แต่มี repository คุยแทน, controller ไม่รู้ว่า repository คุย database ยังไง ให้ service ติดต่อแทน
 - High Cohesion: แยกหน้าที่ชัดเจนระหว่าง class
 - repository(คุยกับ database), service (ตัวกลางและ logic), controller (req ,res)

อธิบาย Execution Flow:

1. Client Request ไปที่ controller (get,post)
2. controller ส่งต่อ req ไปที่ GameService โดยใช้ Dependency Injection
3. GameService จัดการ Business logic และเรียกใช้ repository
4. repository จัดการข้อมูล database โดยใช้ JPA
5. Database จัดการข้อมูล
6. Repository ส่งข้อมูลกลับไป service
7. Service ส่งข้อมูลไปที่ controller
8. Controller ส่งข้อมูลให้ thymeleaf เพื่อจัดการไฟล์ .html
9. Client ได้รับ response เป็นไฟล์ html
10. แสดงข้อมูลให้ client

ส่วนที่ 2: Code Implementation & Explanation

- Entity

Java

```
@Getter // getter and setter ทุก attributes
@Setter
@NoArgsConstructor // เพิ่ม constructor เปล่า
@Entity // ระบุว่าเป็น entity
@Table(name = "games") // ตั้งชื่อ table in database
public class Game {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id; // Primary key
    private String title;
    private String genre;
    private String platform;
    private double rating;
    @DateTimeFormat(pattern = "yyyy-MM-dd") // fixed format date
    private LocalDate releaseDate;
    private double price;
    private String discountType;
    public String getDiscountName() { // return discount name เป็นภาษาไทย
        return new DiscountContext().getDiscountName(discountType);
    }
    public double getFinalPrice() { // return ราคาที่คำนวณแล้ว
        return new DiscountContext().getDiscountPrice(price, discountType);
    }
}
```

- Repository

Java

```
@Repository
public interface GameRepository extends JpaRepository<Game, Integer> {}
```

อธิบาย: JpaRepository เป็น interface สำหรับ จัดการ database เช่น การ query, insert หรือ delete

โดยเหมาะสำหรับการทำ CRUD

- Strategy Package

Java

```
public class DiscountContext {
    // check type of discount
    private DiscountStrategy getStrategy(String discountType) {
        switch (discountType) {
            case "STUDENT":
                return new StudentDiscountStrategy();
            case "SEASONAL":
                return new SeasonalSaleStrategy();
            default:
                return new NoDiscountStrategy();
        }
    }
    public String getDiscountName(String discountType) {
        return getStrategy(discountType).discountName();
    }
    public double getDiscountPrice(double price, String discountType) {
        return getStrategy(discountType).calculate(price);
    }
}
```

อธิบาย: DiscountContext ทำหน้าที่เป็นจุดตัดสินใจเดียวว่าจะใช้ strategy ไหน แล้วมอบหมายการคำนวณจริงให้ strategy แต่ละตัว

- Service

Java

```
@Service // ระบุว่าเป็น service layered arch
public class GameService {
    @Autowired // ใช้ทำ Dependency Injection
    private GameRepository gameRepository;
    public List<Game> getAll() {
        return gameRepository.findAll(); // query ( select * from games; )
    }
    public void saveGame(Game game) {
        gameRepository.save(game); // insert ลง database
    }
    public Game getById(int id) {
        return gameRepository.findById(id).get(); // where id = id
    }
    public void deleteById(int id) {
        gameRepository.deleteById(id); // delete id โดยใช้ where id
    }
}
```

- Controller

Java

```
@Controller // ระบุว่าเป็น controller ; ใช้ RestController เมื่อเป็น RestApi
@RequestMapping("/games")
public class GameController {
    @Autowired
    private GameService gameService; // Dependency Injection
    @GetMapping()
    public String list(Model model) {
        model.addAttribute("games", gameService.getAll());
        return "games/list";
    } // Page List
    @GetMapping("/add")
    public String add(Model model) {
        model.addAttribute("game", new Game());
        return "games/add";
    } // Page Add
    @PostMapping("/save")
    public String save(@ModelAttribute Game game) {
        gameService.saveGame(game);
        return "redirect:/games";
    } // Add
    @GetMapping("/delete/{id}")
    public String delete(Model model, @PathVariable int id) {
        model.addAttribute("game", gameService.getById(id));
        return "games/delete";
    } // Page Delete
    @PostMapping("/delete/{id}")
    public String deleteById(@PathVariable int id) {
        gameService.deleteById(id);
        return "redirect:/games";
    } // Delete By Id
    @GetMapping("/edit/{id}")
    public String edit(@PathVariable int id, Model model) {
        model.addAttribute("game", gameService.getById(id));
        return "games/edit";
    } // Page Edit
    @PostMapping("/update/{id}")
    public String update(@PathVariable int id, @ModelAttribute Game game) {
        game.setId(id);
        gameService.saveGame(game);
        return "redirect:/games";
    } // Edit By Id
}
```

- Dependency Injection (Constructor Injection)
 - การใช้ DI เป็นการรับ object ที่ต้องการ เหมือนเชื่อมต่อจาก layered อื่น เช่น controller เรียกใช้ service โดยจะช่วยลดการสร้าง object

ส่วนที่ 3: Web Application & Database Screenshots

- create

The screenshot shows the 'Game Catalog' web application with the 'Add New Game' form. The form includes fields for game name, platform, genre, rating, price, and release date. The 'Platform' field is set to 'PC, mobile' and the 'Release Date' is set to '12/08/2026'.

Game Catalog

CP33302 - Lab 7

← กลับสู่รายการเกม

🔍 **เพิ่มเกมใหม่**

กรณียกข้อมูลเกมนี้คือการเพิ่มเกมใหม่

📁 ชื่อเกม

roblox 673380065-6

📁 ประเภท (Genre)

sandbox

📁 แพลตฟอร์ม (Platform)

PC, mobile

📁 คะแนน (Rating)

10

📁 ราคาต่อชั่วโมง (Price)

100

📁 ส่วนลด (Discount)

ส่วนลดพิเศษ (10%)

📅 วันวางจำหน่าย (Release Date)

12/08/2026

💾 บันทึก

🗑️ ลบ

- read

The screenshot shows the 'Game Catalog' web application with the 'All Games' list. The list displays 4 games: Minecraft, genshin, ROV, and roblox 673380065-6. Each game entry includes its name, platform, genre, rating, release date, price, and discount.

Game Catalog

CP33302 - Lab 7: Spring Boot + JPA + JDBC + PostgreSQL

🔍 **รายการเกมทั้งหมด**

จัดการข้อมูลเกม/รายการ — Create, Read, Update, Delete

🔍 **เพิ่มเกมใหม่**

#	ชื่อเกม	ประเภท	แพลตฟอร์ม	คะแนน	วันวางจำหน่าย	โปรโมชั่น (STRATEGY)	ราคาต่อชั่วโมง (Price)	ส่วนลด
1	Minecraft	sandbox	PC	★ 10.0	15/08/2026	ราคาต่อ	800.00	🔍 📄
2	genshin	openworld	PC, mobile	★ 8.0	31/07/2026	ส่วนลดพิเศษ 25%	180.00 200.00	🔍 📄
3	ROV	mobile	mobile	★ 10.0	09/08/2026	ส่วนลดพิเศษ 15%	90.00 100.00	🔍 📄
4	roblox 673380065-6	sandbox	PC, mobile	★ 10.0	12/08/2026	ส่วนลดพิเศษ 10%	90.00 100.00	🔍 📄

📄 1 ถึง 4 ของ 4

CP33302 Principles of Software Design - Lab 7: Database Connectivity

- update

The screenshot shows the 'Game Catalog' web application with the 'Edit Game' form. The form includes fields for game name, platform, genre, rating, price, and release date. The 'Platform' field is set to 'PC, mobile' and the 'Release Date' is set to '12/08/2026'.

Game Catalog

CP33302 - Lab 7

← กลับสู่รายการเกม

🔍 **แก้ไขข้อมูลเกม**

แก้ไขข้อมูลเกม: roblox 673380065-6

📁 ชื่อเกม

roblox 673380065-6

📁 ประเภท (Genre)

sandbox

📁 แพลตฟอร์ม (Platform)

PC, mobile

📁 คะแนน (Rating)

10.0

📁 ราคาต่อชั่วโมง (Price)

20000

📁 ส่วนลด (Discount)

ส่วนลดพิเศษ (20%)

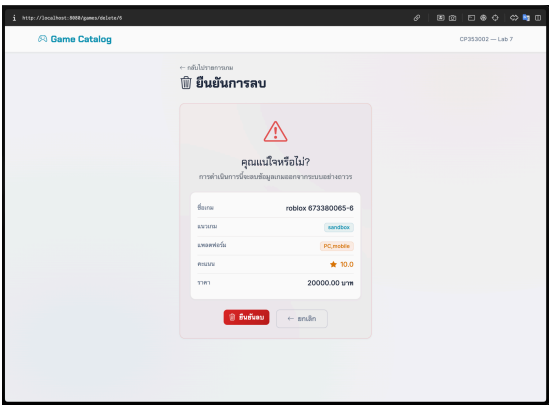
📅 วันวางจำหน่าย (Release Date)

12/08/2026

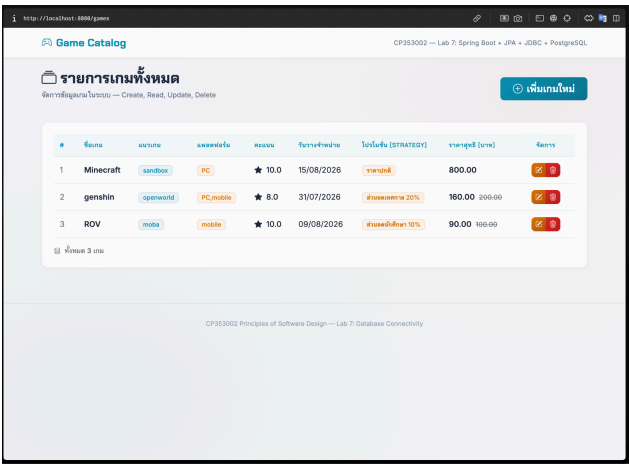
💾 บันทึก

🗑️ ลบ

- delete



- List and database



```
lab7demo=# select * from games;
id | discount_type | genre | platform | price | rating | release_date | title
-----+-----+-----+-----+-----+-----+-----+-----
4 | NONE | sandbox | PC | 800 | 10 | 2026-08-15 | Minecraft
3 | SEASONAL | openworld | PC,mobile | 200 | 8 | 2026-07-31 | genshin
1 | STUDENT | moba | mobile | 100 | 10 | 2026-08-09 | ROV
(3 rows)
```